

DSCI 511 Quiz 2 Cheat Sheet — 内容稿

DataFrame：读入、检查、选择

```
import pandas as pd

df = pd.read_csv("data.csv")
df = pd.read_csv("data.tsv", sep="\t")
df = pd.read_csv("no_header.tsv", sep="\t", header=None,
                 names=["id", "temp"])
df = pd.read_csv("data.csv", index_col="id")

• 先查 df.shape、df.columns.to_list()、df.head()、df.info()、df.describe()；.shape/.columns 是 attribute，没有 ()。
• df["col"] 是 Series；df[["col"]] 是单列 DataFrame。
  s.value_counts() 计频数，s.nunique() 计不同值数。
• df.loc[row_label, col_label] 按 label；df.iloc[row_pos, col_pos] 按从 0 开始的位置，不可传列名。单一选取取降维；用 list（如 df.loc[[key]]) 保留 DataFrame。

df = df.set_index("iso_code")          # 返回新表；须赋值
x = df.loc["aiw", "continent"]
x = df.loc[df.index[6], "continent"]   # 第 7 行位置 + 列 label
x = df.loc["aiw", df.columns[0]]      # 行 label + 第 1 列位置
```

整列计算、排序、过滤

```
• 列运算向量化且按 index 对齐：df["scaled"] = df["score"] * 1.03；不要为逐元素算术写 loop。
• 归约中 axis=0（默认）给每列一个值，axis=1 给每行一个值：df.mean(axis=1)、df.idxmax(axis=1)。

recent = df.sort_values(by="year", ascending=False)
mask = (df["region"] == "Cariboo") & (df["year"] > 2015)
out = df[mask]
out = df.query("region == 'Cariboo' and year > 2015")
```

- mask 用 &、|、~，每个比较各加 ()；不用 Python 的 and、or、not。
- .query() 内直接写列名、可用 and/or/not；含空格的列名用反引号，字符串值用引号。涉及 .isin()、.str... 时 mask 更灵活。

清理、修改与写出

```
has_missing = df.isna().any()
complete = df.dropna()                # 返回新表
filled = df.fillna("Unknown")         # 返回新表

df = df.drop([14379, 54578])           # 按 index label 删行
df = df.drop(["Art", "Drama"], axis=1) # 删列
df = df.rename(columns={"area": "region"})
df.to_csv("clean.csv", index=False)    # 先决定是否写出 index

• 缺失值先问业务含义；NaN 不等于空字符串，也不自动等于 0。长格式资料可因缺少整行而看似没有 NaN；pivot/merge 后缺口才会显现。
• df.columns = [...] 是整体原地改名，长度和顺序必须完整匹配；只改少数列用 rename。
• set_index、drop、fillna、rename、sort_values、reshape、reset_index 常返回新对象：没赋值就没替换原变量。
```

Tidy data 与 reshape

Tidy：每个变量一列、每个 observation 一行、每种 observational unit 一张表。先定义“每行代表什么”，再选 long/wide。

```
wide = df.pivot(index="country", columns="metric", values="value")
means = df.pivot_table(index="language", columns="accent", values="score")
long = df.melt(id_vars=["Director"], value_vars=["Star1", "Star2"],
               var_name="Billing_Order", value_name="Actor")
clean = wide.reset_index()

操作      何时用 / 关键点
df.transpose() 行列互换，返回新表。
pivot      long → wide；每个 (index, columns) 组合必须唯一，否则报错。
pivot_table 组合有重复：默认聚合为 mean，不是随便保留一行。
melt       wide → long；id_vars 保留，原列名和值分别进 var_name/value_name。
reset_index 把 reshape 后的 index 还原为普通列，返回新表。

• pivot 后的 NaN 可能只是不存在的组合；只有语义确为 0 时才 .fillna(0)，再视需要 .astype(int)。
• 省略 melt(value_vars=...) 会 melt 所有非 id_vars 列；保留某些测量列时显式写出。
```

拼接与按 key 合并

```
stacked = pd.concat([w1, w2, w3])      # 默认 axis=0，叠行
side_by_side = pd.concat([w1, w3], axis=1) # 按 index 并列

both = pd.merge(w1, w4, how="outer", on=["Date", "Month", "Year"])
lookup = pd.merge(weather, seasons, how="left", left_on="Month", right_on="month")
```

```
by_index = pd.merge(w1, w4, how="outer", left_index=True, right_index=True)

• concat 是沿轴直接接；它不理解业务 key。列集合不同会生 NaN，重复 index/列仍可能保留。
• merge 按实体 key 匹配：同名 key 用 on，异名 key 用 left_on/right_on，按 index 则两侧 * index=True。outer 保留两侧 key，left 保留左表 key，right 保留右表 key。
• merge 前检查 key 含义与重复；重复 key 可形成 many-to-many，行数暴增是信号。按列 merge 会重建默认 index。
```

apply、map、groupby

写法	函数接收 / 结果
df.apply(func, axis=0)	每列一个 Series；默认每列一个结果。axis=1 时每行一个 Series。
df.map(func)	DataFrame 每个 scalar 各调用一次。
s.apply(func)	对 Series 应用函数。
s.map(dict_or_func)	单值映射；可查 dict。未映射 key 变 NaN。
col_sums = df[["a", "b"]].apply(sum) df2 = df.copy() df2["food_type"] = df2["food"].map(food_type_map)	
by_country = (df.groupby("country")["co2_emmission"].sum().sort_values(ascending=False)) summary = df.groupby("year").agg(mean_score=("score", "mean"), total=("score", "sum"))	

- 函数吃/回传一整列或一整行时用 apply；吃 scalar 时用 map。题目要保留原表时先 .copy()。
- groupby 先 split 成分组对象，再 .sum()/.mean()/.count()/.agg()；多键会形成组合 index。只 aggregate 数值列，避免无关 object 列。要把 group key 变回列时再 reset_index()。

字符串、datetime、category

```
parts = df["Genre"].str.split(",", expand=True)
parts[1] = parts[1].str.strip()

df["Date"] = pd.to_datetime(df["Date"])
ts = pd.read_csv("cycling.csv", parse_dates=True, index_col=0)
ts.sort_index().loc["2019-10-01":"2019-10-13"]
by_day = ts[["Time", "Distance"]].resample("1D").mean()

s = pd.Series(pd.date_range("2020-01-01", periods=3))
years = s.dt.year
df["Class"] = df["Class"].astype("category")
df["level"] = pd.Categorical(df["level"], categories=["low", "medium", "high"], ordered=True)
```

- Series 的文本方法由 .str 进入；拆分后常要 .str.strip() 去分隔符后的空白。
- Timestamp 是时间点；Period 是时间段；Timedelta 是时长；缺失 datetime 是 NaT。pd.date_range/pd.period_range 建时间 index。
- DatetimeIndex 可用部分字符串，如 .loc["2019-10"]；日期范围 slice 先 sort_index()。按日内时刻选 .between_time("00:00", "01:00")。DatetimeIndex 直接 .weekday/.day_name()；datetime Series 用 .dt.year，不是 s.year。
- .resample(freq) 是时间专用 groupby，必须再 aggregate；"1D" 日、"h" 时、"min" 分、"B" 工作日、"MS" 月初、"ME" 月末。空时间桶常为 NaN，是否填值看语义。
- 有限且稳定取值适合 category；它可省内存、加速部分 groupby/sort，并表达非字母业务顺序。先清理文本再转 category，否则大小写/拼写变成不同 level。

快速查错顺序

1. 先写对象类型、shape、index、列名和 dtype。
2. 再问选择按 label 还是 position、需要 Series 还是 DataFrame。
3. 变换后检查是否返回新表、是否已赋值；筛选条件逐项加括号。
4. reshape 先定 observation；merge 先定 key 和基数；datetime 先转换、范围切片先排序。